PROJECT MILESTONE

INFO 290 GENAI - FINAL PROJECT PROPOSAL

Jyoti Divya, Pan Haoming, Lin Iyu, Baier Noah
divya_jyoti@berkeley.edu, haoming.p@berkeley.edu, iylin@berkeley.edu, jnoah_baier@berkeley.edu

MOTIVATION

Proper nutritional management is fragmented across calorie trackers, grocery apps, and manual logging, leading to wasted food, overspending, and missed health goals. **MacroMind** unifies macro-aware meal recommendation, grocery planning, and fridge inventory tracking into a single system. Using a multimodal RAG architecture, it retrieves recipes from datasets such as **RecipeNLG**, nutritional information from **USDA FoodData Central**, and pricing data from publicly available grocery APIs. The system then reasons under macro, budget, and dietary constraints to generate personalized recommendations. A lightweight computer vision module estimates fridge inventory from images, and a personalized memory layer tracks user preferences over time. The system is designed for health-conscious individuals and budget-aware households seeking explainable, constraint-aware nutrition guidance rather than static meal plans.

DATA SOURCE AND INPUTS

Recipes data: The RecipeNLG dataset contains 2,231,142 recipes collected from multiple sources.
Each recipe includes the following fields:

- Unnamed: 0 : row index (dropped during preprocessing)
- Title: recipe name
- Ingredients: raw ingredient strings with quantities and units, like "1 c. firmly packed brown sugar"
- Directions: step-by-step cooking instructions as a list of sentences
- Link: recipe URL
- Source: either "Gathered" (~1.6M, higher quality) or "Recipes1M" (merged dataset, noisier)
- NER: clean ingredient names without quantities or units (e.g. "brown sugar", "milk", "butter"), generated by a Named Entity Recognition model

Preprocessing and data cleaning:

- Using only recipes labeled as Gathered to reduce noise from merged datasets.
- Removing outlier recipes with unusually large numbers of ingredients
- Handling missing values by dropping recipes with missing titles (only 1 instance)
- Using the provided NER field as the standardized representation of ingredients. This removes quantities and units, leaving only normalized ingredient names.
- The cleaned recipes and their NER-based ingredient lists are stored in a structured format (e.g., JSON), which will be used for ingredient normalization and nutrition lookup.

Challenges:

- The NER field is generated automatically and may contain errors, incomplete ingredient names, or inconsistencies (e.g., including brand-like terms).
- NER doesn't contain quantity and unit data, so we parse quantities and units from raw ingredient text.
- Ingredient strings vary widely in format (e.g., "1 c.", "1 cup", "250 ml")
- Due to large-scale scraping, many recipes are duplicates, which may bias future retrieval.

Nutrition data: USDA FoodData Central
It is a free API containing 463,000 food entries, including:

- Foundation (365): lab-verified, highest quality
- SR Legacy (7,793): classic USDA reference, wide coverage
- Branded (454,678): manufacturer self-reported, commercial products

Each food item includes key fields:

- fdcId: unique food id
- description: food name (e.g., "Chicken, breast, boneless, skinless, raw")
- dataType: Foundation / SR Legacy / Branded
- foodCategory: category label
- foodNutrients: nutrient list including protein, fat, carbohydrates, and calories

Preprocessing:

- Unifying different calorie IDs (2047 Foundation, 1008 others) into a single calories field.
- Prioritizing Foundation > SR Legacy > Branded when multiple matches exist.

How we use this data:
With the NER ingredient names from RecipeNLG, we query the USDA API using ingredient names. We then select the best match using a scoring-based approach, based on token overlap and metadata (e.g., dataType and foodCategory), and extract macro values (protein, fat, carbohydrates, calories per 100g).

Challenges:

- RecipeNLG and USDA using different naming conventions (e.g., "brown sugar" vs. "Sugars, brown")
- Nutritional values are provided per 100g, while recipe ingredients use inconsistent units (e.g., "1 cup", "1 c.", "250 ml")

SYSTEM ARCHITECTURE AND VARIANTS (System Architecture Diagram)

MacroMind is a constraint-aware RAG system with four stages: (1) query construction: user constraints (macros, budget, pantry, dietary tags) are serialised into a natural-language string for semantic search; (2) retrieval: all-MiniLM-L6-v2 (384-dim, CPU-only) encodes the query and retrieves top-20 candidates from a ChromaDB collection via cosine similarity; (3) constraint-aware reranking: candidates are re-scored as score = 1 − (0.50×MAPE + 0.30×budget_overshoot + 0.20×waste_fraction), with a hard −999 penalty for dietary tag violations, and the top-5 results are selected; (4) LLM generation: Gemini 2.0 Flash/Groq's Llama 3.3 70B Versatile is prompted with the constraints and top-5 recipe cards to produce a full-day meal plan with macro totals and per-meal rationale.

The three variants are structured as a progressive ablation: each adds one component to isolate its contribution. V1 establishes a parametric baseline with no grounding; V2 adds retrieval to measure the grounding benefit alone; V3 adds constraint-aware reranking on top to test whether explicit multi-objective scoring improves over semantic similarity. A Streamlit UI wraps the full pipeline, allowing live variant switching, on-demand corpus ingestion, and side-by-side recipe card inspection.

Variant	Retrieval	Reranking	Purpose
V1 — Baseline LLM	None	None	LLM parametric knowledge only
V2 — RAG	Semantic top-5	None	Grounding benefit alone
V3 — RAG + Reranking	Semantic top-20→top-5	Constraint-aware MAPE	Full constraint-optimised pipeline

EXPERIMENTS AND EVALUATION

We evaluate V2 and V3 across five user profiles: Body Recomposition (2000 kcal / $12), Muscle Bulk (2800 kcal / $16), Vegetarian Weight Loss (1600 kcal / $9 / vegetarian), Budget Meal Prep (1900 kcal / $7), and Keto / Low Carb (1800 kcal, 30 g carbs / $14 / keto). Six ground-truth relevant recipes were manually annotated per profile. The corpus, embedding model, and prompt template are held constant across variants. Metrics: macro MAPE (mean absolute % deviation across cal/protein/carbs/fat, lower is better), Precision@5 (fraction of top-5 in the ground-truth set, higher is better), and budget adherence (binary).

Test Case	RAG Macro Dev	RR Macro Dev	RAG P@5	RR P@5	Budget
Body Recomp	31.3%	43.1%	0.20	0.60	✓
Muscle Bulk	16.4%	12.0%	0.40	0.60	✓
Vegetarian	20.8%	64.1%	0.20	0.40	✓
Budget Meal Prep	49.5%	61.3%	0.20	0.20	✓
Keto / Low Carb	113.1%	40.0%	0.60	0.80	✓
Mean	**46.2%**	**44.1%**	**0.32**	**0.52**	**5/5**

RAG+Reranking improves mean Precision@5 by 63% (0.32→0.52) and delivers the largest gains for constrained profiles: the keto hard-penalty reduces macro deviation from 113% to 40% by eliminating

non-keto candidates that semantic search surfaces. For standard profiles (Body Recomp, Vegetarian), reranking increases macro deviation because the 65-recipe corpus is too small to offer meaningfully different top-20 candidates. In this regime, reranking reorders by budget and waste fit rather than macro proximity, trading macro accuracy for other constraint satisfaction, which confirms that macro MAPE and P@5 capture different aspects of quality and that corpus size is the primary bottleneck. Scaling to the RecipeNLG corpus (already partially ingested) is the top priority for the next phase. All variants achieve 100% budget adherence across all test cases.

As a planned extension, we hope to implement LLM-as-judge macro extraction to evaluate V1, RecipeNLG corpus scaling, and a small user acceptability study.

CONTRIBUTION

Divya Jyoti & Noah Baier jointly working on the System Architecture and Variants and Experiments and Evaluation sections. Contributions include: designing the three-variant ablation framework (V1–V3), implementing the Streamlit application (app.py), the constraint-aware reranker (src/reranker.py), the LLM generation module (src/llm_generator.py), the ingestion pipeline (ingest.py), experiment design, ground-truth annotation across five user profiles, and results analysis.

Primary notebooks:

- Divya Jyoti: notebooks/divya_evaluation.ipyn
- Noah Baier: notebooks/noah_rag_pipeline.ipynb

Haoming Pan & Iyu Lin : jointly worked on the data processing and nutrition integration components. Contributions include: cleaning and filtering the RecipeNLG dataset, structuring recipes using NER-based ingredient representations, implementing ingredient parsing (quantity and unit extraction), designing and implementing the USDA lookup pipeline, and developing the scoring-based matching method (token overlap + metadata filtering) for ingredient-to-nutrition mapping.

- Haoming Pan: notebooks/haoming_data_pipeline.ipynb
- Iyu Lin: notebooks/iyu_recipe_processing.ipynb